

SQA



Đảm bảo chất lượng phần mềm

Software Quality Assurance



1. Software process models

Nguyễn Anh Hào

Khoa CNTT2, Học viện Công Nghệ BCVT Tp.HCM

✉: nahao@ptithcm.edu.vn

☎: 0913609730

Tài liệu môn học – 2025

Nội dung chính

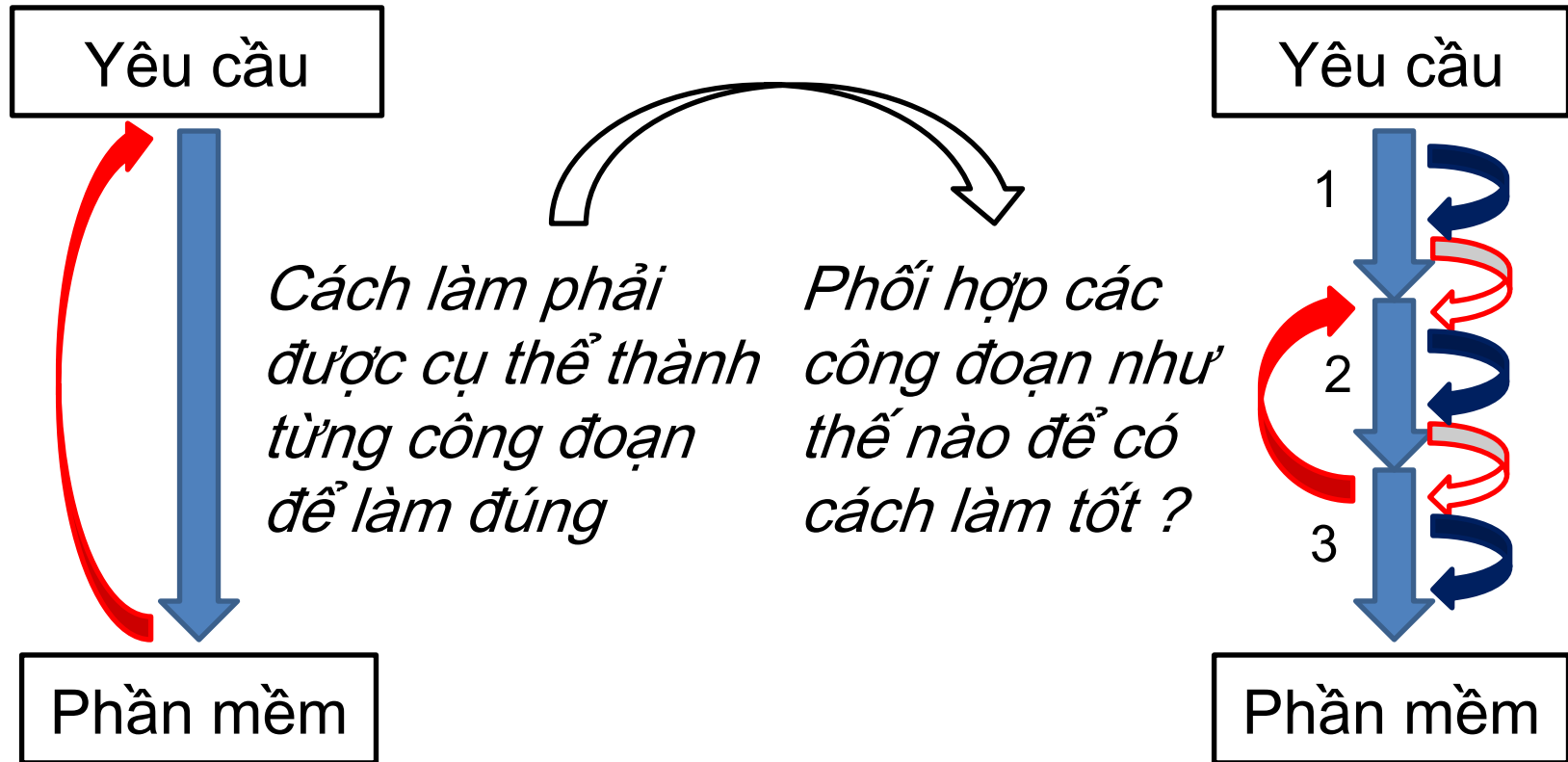
- ❖ Mô hình làm PM là “khuông mẫu” cho các tiến trình tạo ra PM. Trong phần này, chúng ta xem xét “chất lượng” của các mô hình làm PM trong lịch sử.
- ❖ Sự mệnh của PM là thay đổi hoặc tùy biến dễ hơn phần cứng, do đó cách làm PM cũng được xem xét ở khả năng cập nhật phần mềm theo thời gian.
- ❖ Nội dung này giúp chúng ta đánh giá được mức độ thỏa mãn của từng mô hình theo quan điểm trên (ie, xem xét sự tiến hóa của mô hình làm phần mềm).

Câu hỏi khởi đầu

- ❖ Thế nào là một phần mềm có chất lượng ?
- ❖ Làm cách nào để tạo ra một phần mềm có chất lượng ?

Cách làm phần mềm

5



- ❖ Tiến trình làm PM (**SW Process**) là một chuỗi hành động để làm ra (và cải tiến) PM, trong suốt chu kỳ sống của PM.
 - Mỗi phần mềm **chỉ có 1 tiến trình** làm phần mềm.
 - Sự thay đổi của PM thể hiện thành *phiên bản mới* **tốt hơn phiên bản cũ**
- ❖ Mô hình làm phần mềm (**SW process model**) là khuôn mẫu cho tiến trình làm phần mềm; nó định nghĩa các công đoạn và thứ tự thực hiện các công đoạn này
 - Mô hình thác đổ có các công đoạn nối tiếp nhau làm một lần để ra sản phẩm, hoặc làm lại để sửa sai
 - Mô hình xoắn ốc có các công đoạn lặp lại trong nhiều chu kỳ để ra sản phẩm
 - Nó là một hệ sinh thái kết hợp giữa con người (tri thức chuyên gia, thông tin cộng tác), phương pháp luận, phương pháp, kỹ thuật, công cụ và chuẩn.

Một mô hình phát triển PM cần chỉ ra các hành động (các bước) để giải quyết 4 vấn đề:

1. SW requirements

- Định nghĩa yêu cầu cho PM được mong đợi

2. SW design & implementation

- Đưa ra giải pháp: cách thiết kế và hiện thực thành 1 phiên bản PM chạy được

3. SW verification, validation, testing

- Cung cố cho giải pháp thiết kế và phiên bản PM hiện tại để nó thỏa mãn yêu cầu.

4. SW evolution (maintenance)

- Cách cập nhật phiên bản PM hiện tại để nó thỏa mãn cho yêu cầu mới.

Mô hình phần mềm trong lịch sử

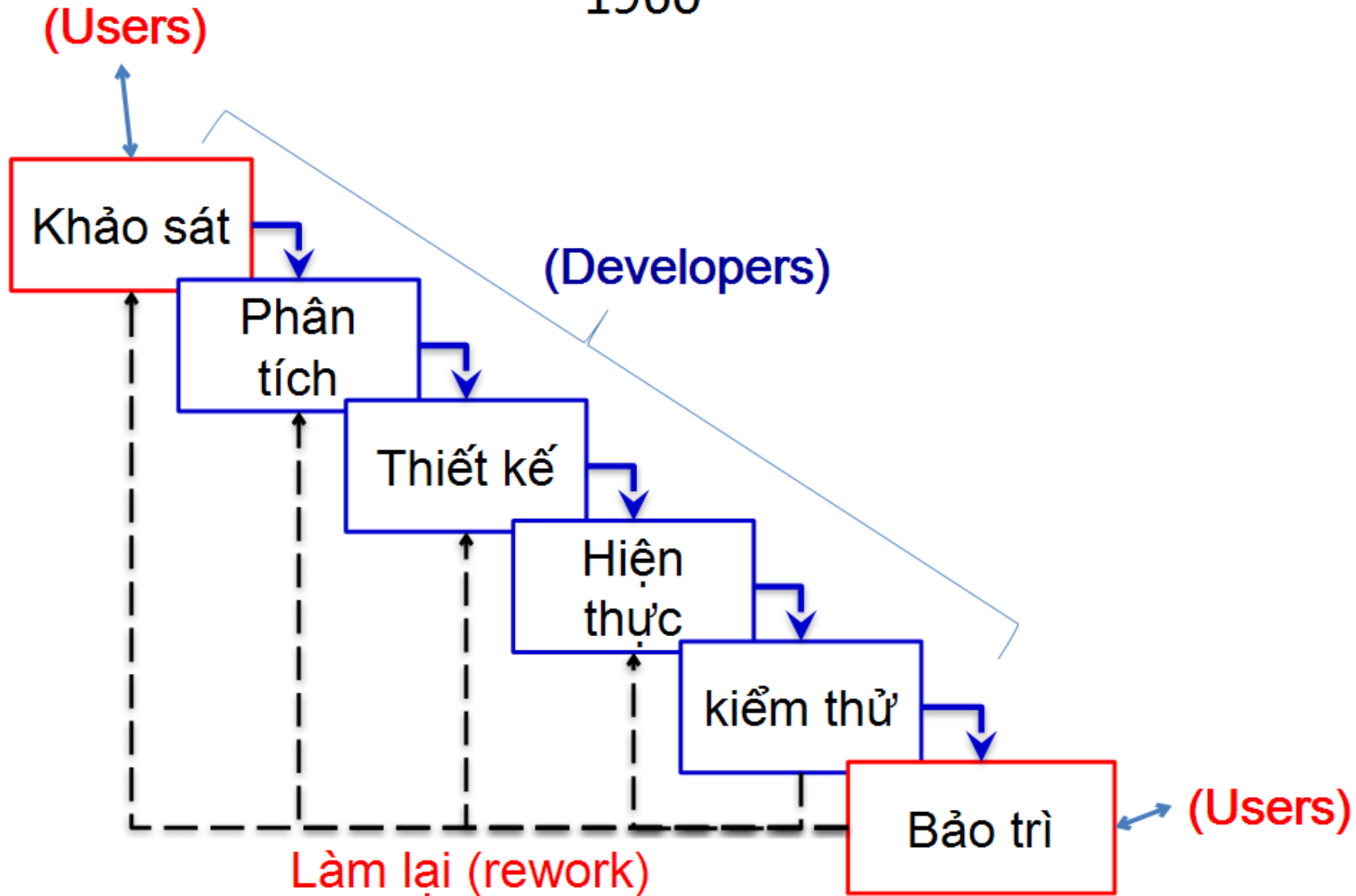
8

- ❖ Trong công nghệ phần mềm (SE), mỗi mô hình được tạo ra tại từng thời điểm lịch sử và để giải quyết những vấn đề khác nhau.
 - Lý do: mô hình phát sinh từ **nhận thức** (pp luận, pp, kỹ thuật) được nâng cấp theo thời gian, dẫn đến có nhiều mô hình mới thay thế cho mô hình đã biết trước đó.
- ❖ Học các mô hình để hiểu biết các đặc trưng và cải tiến của các mô hình để chọn mô hình áp dụng phù hợp với bối cảnh thực tế.
 - Không có mô hình dở, chỉ có cách dùng mô hình dở.

Mô hình thác nước (tuyến tính)

9

1960



- ❖ Mô hình thác đổ (còn gọi là SDLC chuẩn) dựa trên phương pháp luận giải quyết vấn đề có từng bước dẫn dắt quá trình suy nghĩ cách giải quyết vấn đề:
 - Khảo sát : nhận thức **bối cảnh** phát sinh nhu cầu về PM
 - Phân tích: **định nghĩa yêu cầu** cụ thể cho PM
 - Thiết kế: đưa ra giải pháp làm phần mềm “tốt nhất” cho yêu cầu
 - Hiện thực (lập trình): **áp dụng** thiết kế vào thực tế
 - Kiểm thử: **đánh giá** mức độ làm thỏa mãn yêu cầu
 - Bảo trì: **sửa** PM cho phù hợp với yêu cầu mới vừa biết

Đặc điểm : “tuyến tính”

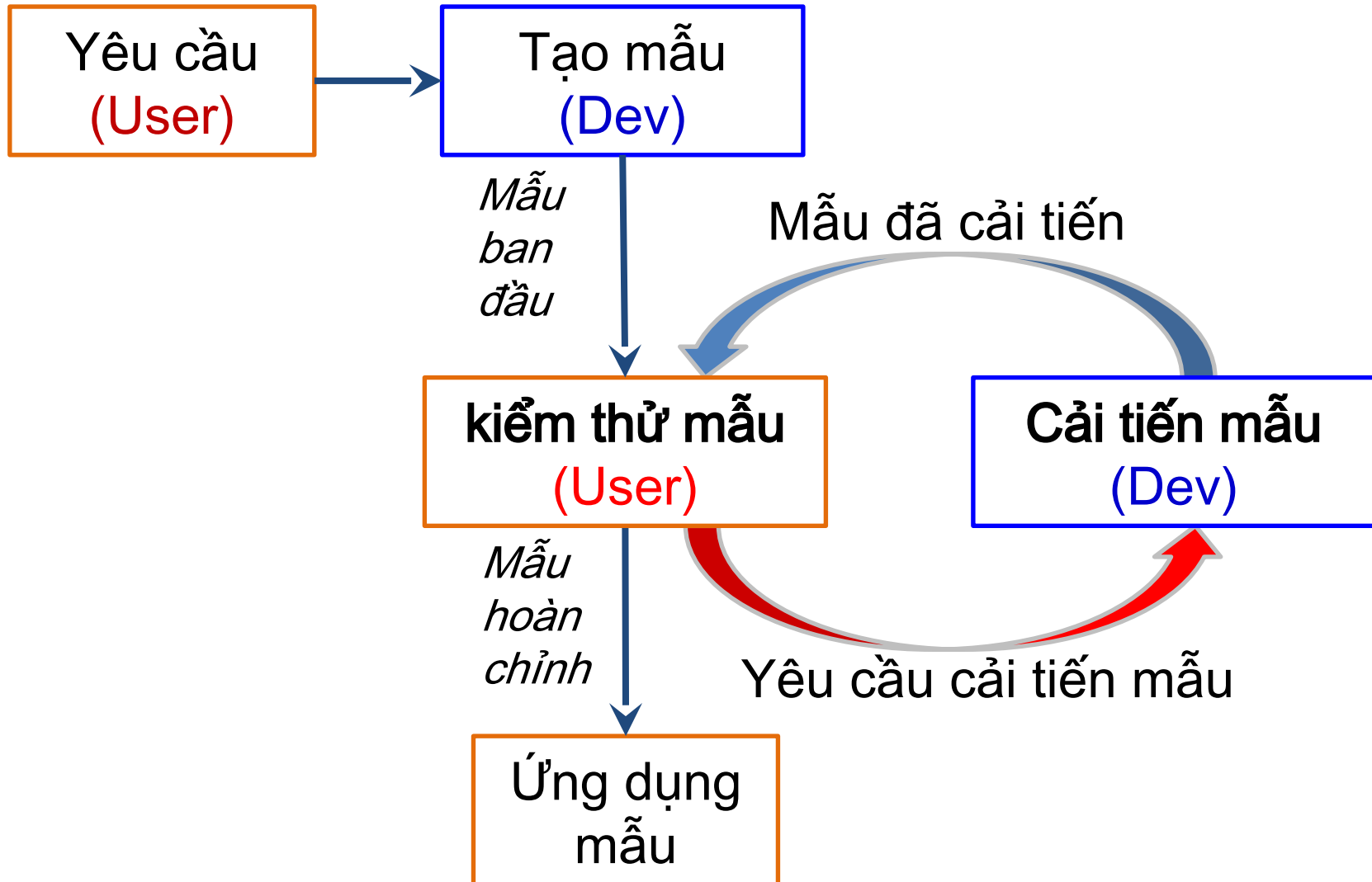
11

- ❖ Có các tài liệu (vd: tài liệu phân tích, thiết kế) để làm công đoạn tiếp theo
 - Công đoạn trước phải hoàn tất để làm công đoạn sau.
- ❖ Người users chỉ đưa ra yêu cầu để có PM sử dụng (không tham gia vào việc làm ra sản phẩm PM)
 - Nếu yêu cầu sai, mọi ấn phẩm phải được xem xét lại.
 - Mô hình này tốn nhiều công sức để làm lại cho các yêu cầu ban đầu bị thay đổi (do rất nhiều lý do)
- ❖ Mô hình này chỉ phù hợp với loại yêu cầu “**chắc chắn đúng, đủ và không thay đổi**”

Mô hình làm mẫu thử

12

1960



Đặc điểm: “trực quan, lặp lại”

13

- ❖ Ngược với mô hình thác đổ, người user (chuyên gia) điều khiển quá trình phát triển mẫu để có một phiên bản mẫu hoàn chỉnh (là PM).
 - Có hợp tác giữa user và dev dựa trên mẫu.
- ❖ Trực quan, dễ nhận xét và bàn luận trên mẫu.
 - User tiếp cận sớm với mẫu (sẽ là sản phẩm PM)
 - Dev hiểu rõ về nhu cầu dùng PM của user.
- ❖ Các công đoạn của SDLC không được yêu cầu
 - Mọi thay đổi đều thể hiện trên mẫu, không có tài liệu.
 - User quyết định mọi chi tiết của mẫu.
 - ✓ nhiều users có thể sinh mâu thuẫn.

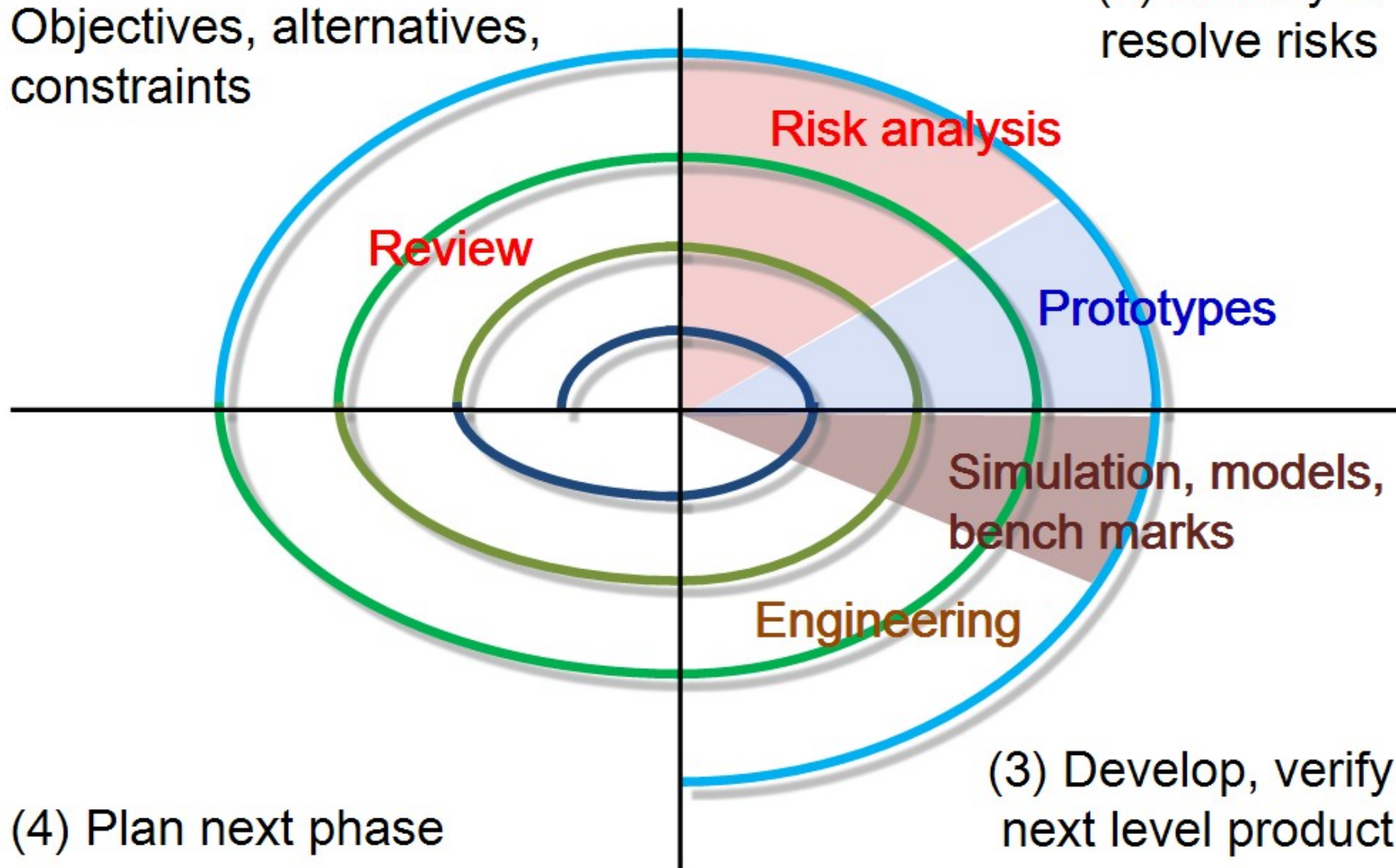
Mô hình xoắn ốc

14

1988, Barry Boehm

(1) Determine
Objectives, alternatives,
constraints

(2) Identify &
resolve risks



❖ Thừa kế các mô hình trước:

- Mô hình thác nước: có các công đoạn để tạo mẫu (có tài liệu ấn phẩm của công đoạn).
- Mô hình mẫu thử: cho phép user tham gia đánh giá & cải tiến mẫu sau mỗi chu kỳ.

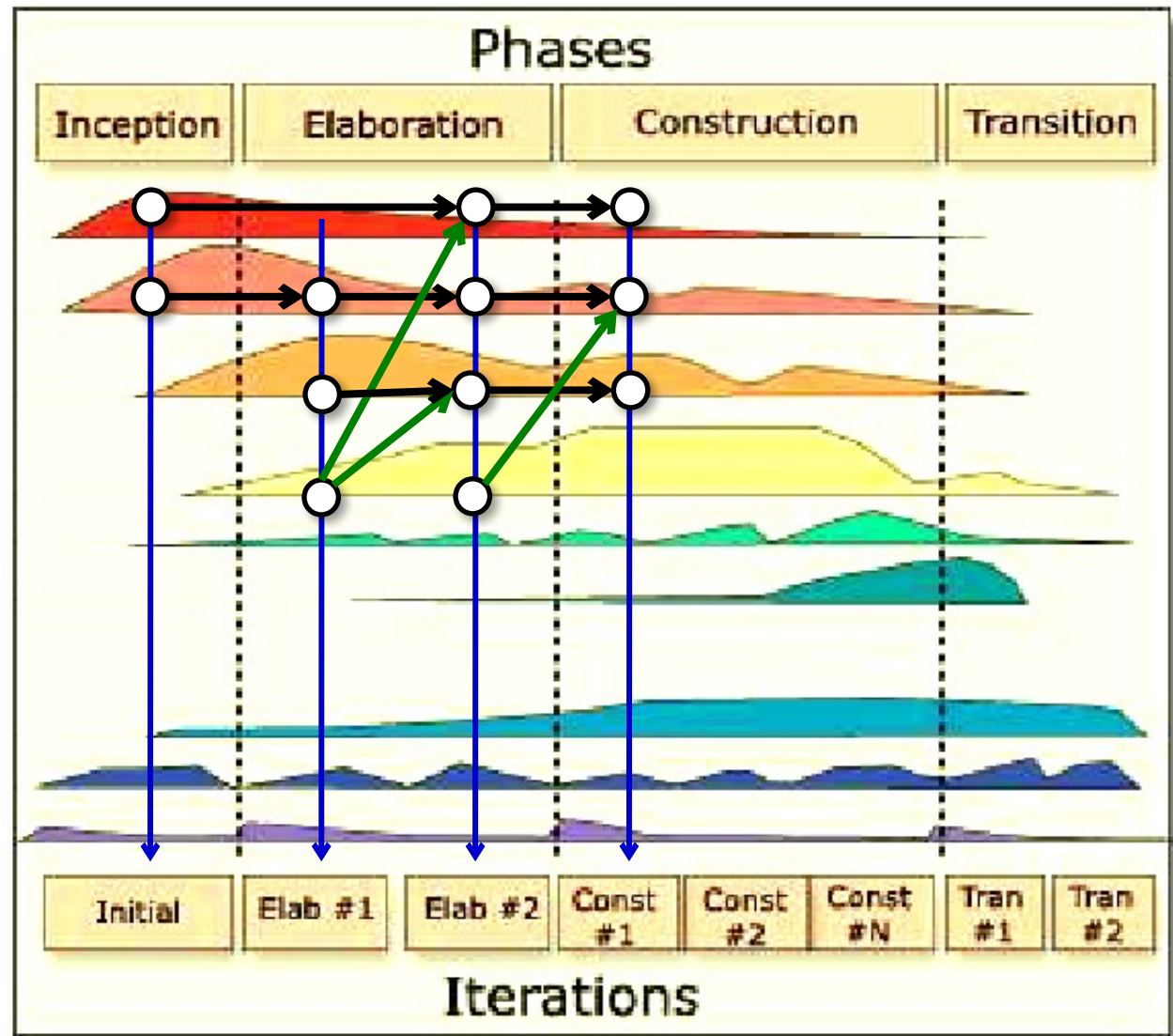
❖ Khác các mô hình trước:

- Làm từ bản chất  chi tiết (bất biến $\leadsto$ tùy biến).
- User := **stake-holders** (User + Dev + Experts + ...)
- Các công đoạn trong chu kỳ: **thay đổi được**.
- Cho phép **cập nhật ấn phẩm**, thay vì tạo mới.
- Cho phép **tiếp tục phát triển** trong khi PM đang được sử dụng.

Mô hình hợp nhất (UP/RUP)

16

1999. Ivar Jacobson. Gradv Booch and James Rumbaugh.



Đặc điểm: “tiến hóa linh hoạt”

17

- ❖ Có các chu kỳ để cải tiến mẫu (giống như xoắn ốc).
 - Mỗi chu kỳ có thể dùng mô hình thác nước, mẫu thử,...
 - Phải sử dụng tiếp cận hướng đối tượng.
- ❖ Mỗi chu kỳ phát sinh công việc cho chu kỳ kế tiếp.
 - Mỗi công đoạn trong một chu kỳ có thể nhận yêu cầu từ bất kỳ công đoạn nào trước đó. vd: kết quả “test” ở chu kỳ trước → sửa “requirements” và bổ sung “design” để thực hiện ở chu kỳ sau.
 - Một công đoạn trở thành một “luồng công việc” cập nhật thường xuyên cho một ấn phẩm.
- ❖ Có 9 luồng công việc làm song song.
 - 6 luồng công việc phát triển mẫu (**engineering**)
 - 3 luồng hỗ trợ phát triển mẫu (configuration & change management, **project management**, **environment**)

Giá trị của mô hình làm phần mềm

18

- 1) Mô hình giúp được gì để có PM “đúng như mong đợi” ?
 - 1 **Xác định rõ yêu cầu trước khi làm**: đặt ra bộ yêu cầu đáng tin cậy cho PM trong môi trường vận hành của nó.
 - 2 **Phân chia công đoạn rõ ràng, dễ kiểm soát**: mỗi công đoạn có vai trò và mục tiêu cụ thể để tập trung nguồn lực.
 - 3 **Tăng cường hợp tác giữa các bên liên quan**: có cơ chế giao tiếp liên tục giữa các chuyên gia: khách hàng, nhóm phát triển, QA, quản lý...
 - 4 **Giảm thiểu sai sót và rủi ro**: có tích hợp rà soát, kiểm thử và đánh giá rủi ro ngay từ đầu, giúp phát hiện sớm lỗi, tránh việc “làm xong rồi mới biết sai”.
 - 5 **Thích nghi với thay đổi**: Waterfall: sửa sai → Spiral: tiến hóa → UP: giám sát môi trường để ứng phó với yêu cầu mới
- 2) Mô hình **không giúp được gì** cho việc tạo ra PM “đúng như mong đợi” ?